# TALLER DE BASE DE DATOS




[ESTUDIANTE : MAGDALENA CORDOVA RAMOS
DOCENTE : SUMIRE
CARRERA : ARQUITECTURA DE PLATAFORMAS TIC
PERIODO ACADEMICO : IV]

[MAGDALENA]

# Sesión 1 — Modelo relacional y consultas

## Pregunta 1

```
SELECT jugador_id, puntos,
 RANK() OVER (ORDER BY puntos DESC) AS ranking
 FROM jugadores;
```

Ranking de jugadores por puntos (sin gaps)

```
SELECT jugador_id, puntos,
(SELECT COUNT(*) + 1 FROM jugadores j2
 WHERE j2.puntos > j1.puntos) AS ranking
 FROM jugadores j1
 ORDER BY ranking;
```

Ranking correlacionado (con RANK denso) usando subconsulta.

## Pregunta 2

```
SELECT g.*
FROM jugadores j
LEFT JOIN ghost_players g ON j.jugador_id = g.jugador_id
WHERE g.jugador_id IS NULL;
```

Jugadores que NO tienen entrada en ghost_players (LEFT JOIN IS NULL).

```
SELECT * FROM jugadores j
WHERE NOT EXISTS (
SELECT 1 FROM ghost_players g
WHERE g.jugador_id = j.jugador_id
)
```

Equivalente con NOT EXISTS. (más eficiente)

## Sesión 2 — Roles y privilegios

Pregunta 3 CREATE VIEW excluyendo email + GRANT y REVOKE.

```
CREATE VIEW vista_usuarios AS
SELECT id, nombre, rol FROM usuarios;
GRANT SELECT ON vista_usuarios TO 'app_user';
REVOKE SELECT ON usuarios FROM 'app_user';
```

## Sesión 3 — Concurrencia

Pregunta 4

Secuencia de deadlock (dos pestañas).

| Pestaña A | Pestaña B |
|---|---|
| 1. BEGIN; | 1. BEGIN; |
| 2. UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1; | 2. UPDATE cuentas SET saldo = saldo - 200 WHERE id = 2; |
| 3. UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2; | 3. UPDATE cuentas SET saldo = saldo + 200 WHERE id = 1; |

→ Error 1213 (Deadlock found when trying to get lock; try restarting transaction) en una de las pestañas.

Explicación: wait circular → T1 espera por T2 y T2 espera por T1 → el motor detecta el ciclo y aborta una transacción.

## 5) Procedimiento modificado (sin FOR UPDATE)

```
CREATE PROCEDURE procesar_pedido (IN pid INT, IN cant INT)
BEGIN
DECLARE stock_actual INT;
SELECT stock INTO stock_actual
FROM productos WHERE id = pid;      -- puede leer un valor
          -- inconsistente si otro tx modificó
UPDATE productos SET stock = stock_actual - cant
WHERE id = pid;
END;                                → race condition posible
                                      (lectura + escritura no atómica)
```

**Nota:**

Sin FOR UPDATE, dos transacciones pueden leer el mismo stock antes de que se actualice → pueden vender de más.

## 6) Simulación de inyección SQL (stacked queries)

**Versión vulnerable (PREPARE / EXECUTE):**

```
PREPARE stmt FROM 'SELECT * FROM usuarios; DROP TABLE usuarios;';
EXECUTE stmt; DEALLOCATE PREPARE stmt;      -- stacked query:
                                               2da sentencia se ejecuta
```

**Versión segura (con preparación real y un solo statement):**

```
PREPARE stmt FROM 'SELECT * FROM usuarios WHERE id = ?';
SET @uid = 1;    -- parámetro separado, no concatenado
EXECUTE stmt USING @uid;                    } -- solo 1 consulta,
DEALLOCATE PREPARE stmt;                      parámetro no
                                              interpretable como
                                              SQL
```

## 7) CHECK vs TRIGGER (anti-cheat)

CHECK constraint (declarativo, más simple):

```
ALTER TABLE productos ADD CHECK (stock >= 0);
```

TRIGGER (con SIGNAL SQLSTATE para rollback + mensaje):

```
CREATE TRIGGER tg_prevenir_stock_negativo BEFORE UPDATE ON productos
FOR EACH ROW
BEGIN
IF NEW.stock < 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Operación bloqueada: stock negativo no permitido';
END IF;
END;                                -- 45000 = error de aplicación
                                       fuerza rollback
```

Pregunta 8
ALTER TABLE ADD PARTITION p_asia +
INSERT and EXPLAIN.
- ALTER TABLE ventas ADD PARTITION (
  PARTITION p_asia VALUES LESS THAN
  ('Asia'))
- INSERT INTO ventas VALUES (...
  'Asia', ...);
- EXPLAIN SELECT ... FROM ventas
  WHERE region = 'Asia'; → busca solo
  en la partición p_asia.

## Sesión 6 — Respaldos

Pregunta 9
destructive UPDATE + PITR restore
steps with binlog.
1. UPDATE ... SET total = 0 WHERE id = 999;
   (error, sin WHERE → destroza datos)
2. Ver binlog: mysqlbinlog --start-
   datetime='2024-05-20 10:00:00' binlog.000123
3. PITR: restore full backup + replay binlog
   hasta justo antes del UPDATE (posición X).
4. mysqlbinlog ... --stop-position=X | mysql -u root -p
→ datos recuperados al estado anterior.

Pregunta 10
trying DROP TRIGGER as app user and
missing privileges explanation.
- Intenté: DROP TRIGGER audit_log_trg;
- Error: ERROR 1419 (HY000): You do not have
  the SUPER privilege and binary logging is
  enabled (you might want to use the less safe
  log_bin_trust_function_creators variable)

→ El trigger es un objeto con cambios y el
usuario app_user no tiene SUPER. Se necesita
GRANT SUPER ON *.* TO 'app_user'; (con cuidado)

Conclusión:
Fragmentación mejora lecturas, los respaldos y PITR
salvan de errores, y los privilegios existen por una razón.